

CV

Miloš Vasić

Ingénieur AI — LLM, agents autonomes et gouvernance pour en garantir la fiabilité.

- Email : milos85vasic@gmail.com
 - Web : <https://milosvasic.ru> · <https://vasic.digital>
 - GitHub : [vasic-digital](#) · [HelixDevelopment](#) · [Server-Factory](#)
-

Résumé

Ingénieur AI/software construisant des systèmes de développement de bout en bout — depuis les infrastructures LLM multi-fournisseurs et les agents autonomes jusqu'aux couches de QA et de gouvernance qui en assurent l'intégrité. Je ne livre pas de démonstrations ; je livre des plateformes. Plus de 15 ans d'expérience en ingénierie professionnelle (depuis 2009), couvrant les SDK mobiles, l'intégration matérielle en temps réel et les backends distribués, qui convergent aujourd'hui vers un seul objectif : rendre le développement AI autonome fiable à grande échelle.

J'architecture des flottes, pas des monolithes — de grandes applications produits reposant sur des dizaines de petits modules découplés, testés indépendamment, chacun héritant d'un Constitution d'ingénierie partagé et vérifié par une discipline de QA anti-biais, fondée sur des preuves tangibles. Langage principal : Go, avec Kotlin/KMP, TypeScript/React, Python, Swift et Shell. Principe directeur, appliqué mécaniquement plutôt que simplement énoncé : une fonctionnalité n'est achevée que lorsqu'un utilisateur réel peut l'utiliser et qu'il existe des preuves capturées pour le démontrer.

Ce que j'apporte : la capacité de transformer une capacité AI, depuis une idée de recherche jusqu'à un système gouverné, auto-vérifiant et prêt pour la production — un routage LLM qui prouve que chaque modèle fonctionne avant de lui faire confiance, des agents qui débattent et parviennent à un consensus plutôt que de

deviner, des couches de mémoire et de RAG qui ne perdent pas le contexte, et un écosystème entier conçu pour que « *les tests sont au vert* » ne puisse jamais signifier silencieusement « *la fonctionnalité est défectueuse* ».

Compétences clés

- **Systèmes AI / LLM** : abstraction d'infrastructure multi-fournisseurs (40+ fournisseurs), intégration d'outils MCP, RAG, bases de données vector et embeddings, orchestration d'agents (agents CLI headless, workflows en graphe, débats/consensus multi-tours), planification (HiPlan/MCTS/Tree-of-Thoughts), LLMops, benchmarking (SWE-bench/HumanEval/MMLU), vérification LLM, garde-fous défensifs LLM, vision par ordinateur + vision LLM.
- **Ingénierie backend** : Go (Gin), gRPC + Protobuf, HTTP/3 (QUIC), WebSockets, systèmes distribués (incluant spécifications formelles TLA+), services REST à haut débit et workers concurrents.
- **Données** : PostgreSQL, SQLite, SQLCipher (chiffrement au repos), Redis, Neo4j, ClickHouse, stockage d'objets (MinIO/S3/GCS/Azure).
- **Frontend / multiplateforme** : TypeScript/React (Tailwind, Redux Toolkit, i18next), Angular, Electron, React Native, Kotlin Multiplatform, Android/Android TV (Kotlin), iOS (Swift), Tauri/Rust.
- **Infrastructure / DevOps** : Docker & Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels ; CI/CD via GitHub Actions, Gradle, Make.
- **QA / ingénierie de la qualité** : QA anti-biais fondée sur des preuves (HelixQA), harnais de tests avec portes de mutation, go test -race, tests de régression visuelle, tests sur appareils ADB, SonarQube, analyse de sécurité (semgrep/gosec/trivy/snyk/gitleaks/nancy).
- **Gouvernance de l'ingénierie** : Constitution en tant que sous-module ; portes d'héritage et de propagation ; discipline de documentation/couverture sur une flotte de plus de 140 dépôts.

Contenu

Projets sélectionnés

Gouvernance et assurance qualité

- **HelixConstitution** — un recueil universel de règles d'ingénierie distribué sous forme de sous-module Git et hérité par plus de 140 dépôts : des lois techniques déployées et

versionnées exactement comme du code. Une simple mise à jour du sous-module actualise les règles pour l'ensemble du parc, et les mécanismes de propagation vérifient littéralement chaque dépôt consommateur à l'aide de *grep* pour s'assurer de la présence de la clause requise — chaque mécanisme étant couplé à un méta-test de mutation qui prouve que le contrôle lui-même n'est pas une supercherie. Il transforme les « bonnes pratiques que les gens espèrent suivre » en un corpus de règles héritées, auditable et appliqué mécaniquement, éliminant toute possibilité de bluff.

- **HelixQA** — une orchestration d'assurance qualité anti-bluff (Go) fondée sur une règle intransigeante : le critère n'est pas « les tests passent », mais « les utilisateurs peuvent utiliser la fonctionnalité ». Elle exécute des banques de tests YAML rédigés et des sessions QA LLM entièrement autonomes, basées sur la vision, qui ouvrent l'application réelle, vérifient chaque fonctionnalité documentée, traquent les bugs non documentés sur Android/Android TV/Web/Bureau, et refusent d'accorder un PASS sans preuves d'exécution capturées — captures d'écran, logcat, vidéos, traces de pile — accompagnées de tickets AI prêts à être corrigés.

Développement AI et infrastructure LLM

- **HelixAgent** — un service LLM d'ensemble de niveau production (Go/Gin) qui refuse de faire confiance à un seul modèle : il diffuse une requête auprès de plusieurs fournisseurs, organise un débat structuré en plusieurs tours (Proposition → Critique → Revue → Synthèse), et achemine les résultats en fonction de scores de vérification en temps réel, avec un repli progressif — le tout derrière une API API compatible OpenAI, dotée d'une couche de données haute disponibilité, d'une observabilité et de garde-fous. *Go, Gin, PostgreSQL, Redis, Prometheus/Grafana/OpenTelemetry, MCP, Neo4j/ClickHouse/Kafka*.
- **HelixCode** — une plateforme de développement AI distribuée qui découpe le travail en tâches intelligentes, conscientes des dépendances, réparties sur une flotte de workers gérée par SSH, puis effectue des points de contrôle et des retours en arrière pour ne jamais rien perdre en cas d'interruption ; sélection des modèles adaptée au matériel et cycle complet planification/construction/test/refactorisation derrière REST/CLI/TUI/MCP. *Go, Gin, PostgreSQL, Redis, SSH, MCP, llama.cpp/Ollama*.
- **HelixLLM** — un binaire unique, six modes de déploiement : inférence compatible OpenAI et Anthropic sur HTTP/3, évolutive d'un ordinateur portable à un cluster multi-hôtes, avec inférence locale via llama.cpp (CUDA/Metal/ROCm) et une chaîne de repli cloud auto-découvrante, notée par vérification, qui dégrade toujours vers un modèle local garanti. *Go, HTTP/3 QUIC, gRPC/SSE/Kafka, llama.cpp*.

- **LLMProvider / LLMOrchestrator / LLMsVerifier** — l'épine dorsale de l'infrastructure LLM : une interface unique pour 43 fournisseurs avec disjoncteurs, surveillance de l'état de santé, nouvelles tentatives avec *backoff* et *jitter*, et une découverte de modèles honnête (sans repli codé en dur) ; un plan de contrôle thread-safe qui lance et pilote des agents CLI headless (OpenCode, Claude Code, Gemini, Junie, Qwen Code) via un protocole hybride pipe+fichier ; et une source de vérité de vérification dont la porte obligatoire « Vois-tu mon code ? » garantit que seuls les modèles prouvés fonctionnels sont jamais marqués comme utilisables ou exportés.
- **HelixMemory / HelixSpecifier** — un moteur unifié de mémoire cognitive fusionnant quatre backends de pointe (Mem0, Cognee, Letta, Graphiti) derrière une seule interface, avec recherche parallèle et reclassement inter-sources ; et un moteur de fusion pour le développement piloté par spécifications, dont la cérémonie s'adapte à l'ampleur du travail et étaye les spécifications par un débat multi-agents.
- **HelixTrack** — une alternative JIRA + Confluence du monde libre (figure de proue de la gamme Helix-Track) : des microservices Go avec une API unifiée à routage d'actions sur HTTP/3, chiffrement SQLCipher au repos, et des clients natifs Web/Bureau/Android/iOS.

Contenu

Outils de qualité professionnelle (utilitaires vasic-digital)

- **Catalogizer** — gestion de collections multimédias multi-protocoles, chiffrée et auto-hébergeable (Go/Gin + React), résistante aux stockages réseau instables, construite sur 21 sous-modules réutilisables.
- **Courses-Creator** — pipeline de conversion Markdown vers vidéo AI avec TTS et lecteurs pour ordinateur, mobile et web.
- **VisionEngine** — perception d'interface LLM par vision par ordinateur et fournisseurs multiples, intégrant des graphes de navigation.
- **DocProcessor · Docs Chain · Herald · task_bridge · Vasic Digital Suite de modules réutilisables** — cartographie des fonctionnalités QA, synchronisation de documents/bases de données par hachage de contenu, notifications en langage naturel, synchronisation de tâches/tableaux, et la flotte de bibliothèques standard `digital.vasic.*`.

Automatisation de l'infrastructure (Server Factory)

- **Mail Server Factory** — JSON déclaratif → serveurs de messagerie entièrement provisionnés et Dockerisés, couvrant

12 types de connexions et 25 distributions Linux ; rapporte 439 tests réussis et une porte SonarQube validée.

- **Cadre de base Server Factory, Qemu-Utils, Parallels-Utils** — moteur de provisionnement partagé et outils de création d'images VM.

Langages et outils (liste rapide)

Go · Kotlin · Kotlin Multiplatform · TypeScript · JavaScript · Python · Swift · Java · Rust · Shell · PL/pgSQL · TLA+ · Gin · gRPC · HTTP/3 · React · Angular · Electron · React Native · PostgreSQL · SQLite · SQLCipher · Redis · Neo4j · ClickHouse · Docker · Kubernetes · Prometheus · Grafana · OpenTelemetry · QEMU · GitHub Actions · Gradle · Make

Expérience

Ingénieur logiciel depuis 2009, couvrant l'intégralité du cycle de développement — planification, développement, direction d'équipe et déploiement. L'historique complet ci-dessous est issu du dossier vérifié du candidat (milosvasic.ru).

Postes à temps plein

- **Développeur SDK — Harness** (harness.io), Belgrade, Serbie · 03/2020 - 12/2024. Développeur principal sur la famille de SDK pour la division Feature Flag de l'entreprise, axée sur toutes les principales plateformes mobiles et au-delà. Clients et partenaires : AWS, Google et diverses banques. *Technos : Android, iOS, Flutter, React Native, TypeScript, JavaScript, Java, Kotlin, Swift, Go, Ruby.*
- **Ingénieur logiciel — Leica Geosystems** (leica-geosystems.com), Heerbrugg, Suisse · 02/2016 - 02/2020. Développement principalement sur iOS et Android pour les scanners 3D de pointe de Leica Geosystems — communication en temps réel avec le matériel, traitement des données et synchronisation. Partenaire : Autodesk. *Technos : Android, iOS, Java, Kotlin, Swift, C++.*
- **Développeur SDK — Bosch** (bosch.rs), Belgrade, Serbie · 01/2010 - 01/2016. Développeur principal SDK pour le projet Véhicules Connectés SDK — communication Bluetooth en temps réel avec le bus OBD2, traitement et persistance haute performance des données. *Technos : Android, Java, Kotlin.*

Contenu

Autres engagements

- **TN-TECH** (tn-tech.co.rs), Novi Sad, Serbie · temps partiel, depuis 03/2017. Travail pour Globex Data (Canada et Suisse) — Sekur (SekurMessenger), SekurMail, SekurSuite — et la plateforme BusRide. *Technologies : Android, Java, Kotlin, C++, Qt.*
- **Increment Loop** (incrementloop.com), Belgrade, Serbie · temps partiel, depuis 09/2023. L'application Yuno. *Technologies : Android, Kotlin.*
- **Projets open source / organisations personnelles** — HelixTrack, Server Factory (Mail Server Factory, Parallels-Utills, Qemu-Utills), et Vasic Digital (Android-Toolkit, Network-Binder), détaillés dans la section Projets sélectionnés ci-dessus.

Publications

- **Fondamentaux du Kotlin** — auteur auto-édité ; dernière édition révisée en septembre 2022 (*Fondamentaux du Kotlin*, 3^e édition). A également écrit pour Packt Publishing (Royaume-Uni).

Formation

- **Master en Technologies de l'Information Contemporaines** — Université Singidunum, Belgrade, Serbie · 2014.
- **Licence en Informatique et Calcul** — Université Singidunum, Belgrade, Serbie · 2008.